

AlphaPi-One 板卡反向工程报告

数据来源：设备自带 `.py` 源码 + 反编译 `.mpy` (MicroPython mpy v6) + 在线实测 (SoftI2C 扫描 / 寄存器只读探测)。

工具脚本见文末「方法学」。

1. 芯片 / 固件

MCU	ESP32, 主频
UID	<code>b'p\x04\x1d\xb3\x82\x1c'</code>
运行时	MicroPython (<code>.mpy</code> 版本)
内置模块	<code>network / esp / esp32 / bluetooth / btree / framebuffer / uctypes / neopixel / dht</code>
缺失模块	<code>urequests / umqtt / sdcard</code>

2. 启动链

```
boot.py    → protocal.uart_read(0,1)[3]; 若 b&0x0C==0x0C 则 import factory_reset; static_buf=bytearray(40960)
main.py    → import ht_main; ht_main.Start(static_buf)
ht_main.py → controlBoardAlphaPiOne.init()
           InitBackground_buf(static_buf) / play_record_loop()
           主循环: Update() / next(soundLoop) / Loop1() (开热点 aiphapione/12345678)
```

3. 通信总线引脚映射（已确认）

	<code>TX=GPIO3</code> , <code>RX=GPIO0</code>	100ms, timeout	反编译 <code>machine.UART(1,460800,tx=3,rx=0)</code>
半用 I2C	<code>SDA=8</code> , <code>SCL=9</code>	SoftI2C 100 kHz	<code>Init_I2c('car')</code> + 实测扫到 <code>0x20</code>

旧加速度计 I2C	<code>SDA=6</code> , <code>SCL=7</code>	SoftI2C	实测无设备
手册12864 I2C	<code>SDA=4</code> , <code>SCL=5</code>	SoftI2C	实测无设备
SPI 屏 (ST7735)	<code>SCK=41</code> <code>MOSI=42</code> <code>MISO=45</code> <code>CS=40</code> <code>DC=39</code> <code>RST=38</code>	硬件 20 MHz mode0	反编译确认
NeoPixel (WS2812)	<code>GPIO17</code> (3 颗)	<code>neopixel.NeoPixel(Pin(17),3)</code>	反编译确认
震动马达	<code>GPIO4</code>	<code>WritePwm(4, v)</code>	反编译确认
电源/使能	<code>GPIO37</code> 输出高	<code>Pin(37,OUT).on()</code>	反编译确认
遥控摇杆 X	<code>ADC</code> <code>GPIO8</code>	<code>1023 - ReadAdc(8)</code>	反编译确认
遥控摇杆 Y	<code>ADC</code> <code>GPIO7</code>	<code>1023 - ReadAdc(7)</code>	反编译确认
遥控电位器	<code>ADC</code> <code>GPIO6</code>	<code>ReadAdc(6)</code>	反编译确认
遥控按键 ×4	<code>GPIO</code> <code>10/11/12/13</code>	<code>ReadPin(p, PULL_UP)</code> , 低有效	反编译确认
转向舵机	由主程序传入的 PWM 引脚	<code>machine.PWM</code> @50 Hz, 500–2500µs	反编译确认 (<code>servo_pin_map</code> 缓存)

板子一律用 `machine.SoftI2C` (软件 I2C, 任意 GPIO)。用硬件 `I2C(0, ...)` 会报 `invalid pin` 甚至卡死总线。
`scan.py` 中已知总线脚集合: `{0,3,8,9,38,39,40,41,42,45}`。

4. SPI 屏 (ST7735) 引脚

```
spi = SPI(2, baudrate=20000000, polarity=0, phase=0,
          sck=Pin(41), mosi=Pin(42), miso=Pin(45))
tft = TFT(spi, 40, 39, 38)    # 驱动类 = ST7735, 签名 TFT(spi, cs, dc, rst)
tft.initr(); tft.rgb(True); tft.fill(TFT.BLACK); tft.rotation(1)
```

SCK	
MOSI	42
MISO	
CS	40

DC	
RST	38

初始化前 `Pin(37, OUT).on()` 拉高 GPIO37（疑似屏/板载电源使能）。

`43 / 44` 并非屏脚，属于 `pin_map`（`ReadPin/WritePin` 的 GPIO 映射表）。

5. 车用 I2C 设备 `0x20` —— 3 轴电机控制器（深挖）

5.1 结论

`0x20` PCF8574 之类的简单 GPIO 扩展器，而是：固件用 `i2c.writeto_mem()` / `readfrom_mem()` 读写寄存器文件；寄存器按电机分块；能回读并做 PID；实测 `readfrom_mem` 对寄存器寻址有效（PCF8574 不支持寄存器寻址，可排除）。

5.2 初始化（`autoMotionOne.mpy`）

```
CAR_ADDR = 32 # 0x20 控制器 I2C 地址
i2c = controlBoardAlphaPiOne.Init_I2c('car') # SoftI2C(scl=Pin(9), sda=Pin(8), freq=100000)
if CAR_ADDR not in i2c.scan(): # 找不到 0x20 → 回退 UART 模式
    i2c = None
    Pin(8, OUT, value=1)
    uart = UART(2, baudrate=38400, tx=Pin(8), rx=Pin(9), timeout=50)

MOTOR_L = 32 # 0x20 左电机寄存器块基址
MOTOR_R = 16 # 0x10 右电机寄存器块基址
MOTOR_Z = 48 # 0x30 Z 轴（举升/关节）寄存器块基址
```

关键：同一对引脚（8/9）既是 I2C，也是 UART 回退。0x20 不存在时改走 `UART2 @ 38400`（tx=8, rx=9）。

5.3 寄存器映射（每个电机一块，基址 0x10 / 0x20 / 0x30）

<code>+0x00</code>	W	int32 LE（I2C 模式再追加 4 字节 校验）	目标速度（限幅 ±1024）	<code>set_speed(motor, speed)</code>
<code>+0x04</code>	W	int16 LE（限幅 ±1024）	原始功率/占空	<code>set_raw_power(motor, power)</code>

+0x08	R	int32 LE + 4 字节取反	编码器位置 (有符号)	get_position(motor)
+0x0C	W	int16	模式 / 使能	set_ON_OFF(motor, mode)、 speed_mode(l,r)

要点:

- writeto_mem(device, addr, data): device 恒为 CAR_ADDR; addr = 电机基址 + 偏移; 失败重试 (OSError → sleep 10ms → print('write error')) 。
- : set_speed 在 I2C 模式写 speed.to_bytes(4, 'little') + (~speed).to_bytes(4, 'little'); get_position 读回 8 字节后校验 data[0:4]==data[4:8], 再 int.from_bytes(..., signed=True) 。
- position_mode() == speed_mode(True, True); stop(motor) == set_speed(motor, 0) 。

5.4 上层 API (autoMotionOne)

```
speed_mode, set_speed, set_raw_power, set_ON_OFF, stop,
position_mode, get_position, position_pid_poll, demo_run_to, stop_mission,
clamp, set_power, set_all_power, stop_motor, go_distance, go_distance_async,
turn_round, turn_round_async, set_claw,
NP_write, BrightnessUp, SetBrightness, UpdateBrightness, GetBrightness,
setPixelColor / setPixelColorWithoutWright / packRGBd / unpackR|G|B
```

即: , 另含一组 WS2812 灯效控制。

5.5 全量寄存器只读实测 (probe_car.py)

```
scan = ['0x20']
0x00-0x3F 逐字节读: 全部 = 00 (空闲状态)
0x28 连续 6 次读: 稳定 = 00000000
特殊: @0x20 (L 速度) 4 字节读 → ETIMEDOUT ; @0x24 (L 功率) 2 字节读 → ENODEV
(同样地址改成逐字节读则返回 00, 属总线轻微抖动/多字节读时序问题)
```

结论: 0x20 在线且寄存器寻址有效; ; 速度/功率命令寄存器偏“写优先”, 多字节读不稳定——与固件寄存器模型一致。

5.6 未定性

具体芯片型号无法从固件与只读探测 100% 确定 (无原理图) 。特征: I2C 0x20、3 个 16 字节寄存器块、速度/PWM/位置/模式寄存器、位置带回读与取反校验、可切 UART@38400 —— 属该车专用的 。

6. 车体链路与功能模块（反编译）

<code>controlBoardAlphaPiOne.mpy</code>	板级底层：UART/I2C 初始化、 <code>ReadPin/WritePin/ReadAdc/WritePwm/SetPwm</code> 、 显示、音频、WiFi/UDP、Update 主循环	<code>Init_I2c</code> ； <code>Update</code> 里 <code>uart_read(0,1)</code> 读 22 字节状态		
<code>autoMotionOne.mpy</code>	车体运动：3 轴电机 + 夹爪	0x20 I2C（UART2@38400 回退）		
<code>steeringEngineActuatorAlphaPiOne.mpy</code>	转向舵机	<code>machine.PWM</code> @50 Hz， ClampD(500,2500)，角度 0~180°→ 脉宽 <code>v/180*2000+500</code>		
<code>remoteControlSensorOne.mpy</code>	遥控/摇杆/按键采集	<code>ReadAdc(8)=X</code> ， <code>ReadAdc(7)=Y</code> ， <code>ReadAdc(6)</code> =电位器；按键 <code>GPIO</code> <code>10/11/12/13</code> （PULL_UP 低有效）；产出 15 项 <code>status_list</code>		
<code>remoteControlActuatorOne.mpy</code>	RGB 灯效 + 震动	<code>neopixel.NeoPixel(Pin(17),3)</code> ； <code>WritePwm(4, v)</code> 震动马达； <code>`hsl/packRGB/unpackR</code>	G	B`
<code>max30102.mpy</code>	通用 MAX30102 心率/血氧驱动类	构造 <code>(i2c, i2c_hex_address)</code> ， 默认 0x57；车用总线上未扫到		
<code>ST7735.mpy</code> / <code>sysfont.mpy</code> / <code>basic.mpy</code>	屏驱 / 字体 / 基础类(DataStruct 等)	—		

7. 其它 I2C 总线（当前未挂设备）

旧加速度计	SDA=6, SCL=7	实测 <code>[]</code>
手册12864 LCD	SDA=4, SCL=5	实测 <code>[]</code> （GPIO4 亦被用作震动马达 PWM）

8. ADC（`ATTN_11DB`，`read_u16`）

- 可用：`GPIO 1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18`
- 不可用：`GPIO0`（strapping）、`GPIO19-39`（32-39 为 ADC1，受 WiFi/BLE 占用而失败）

9. GPIO 上电电平（参考）

- 读为 1：GPIO0, 26, 34, 35, 37, 38, 39
- 读为 -（取值抛异常，多为被固件配成输出/外设）：GPIO19,20,22,23,24,25,27,28,29,30,31,32,33

10. 设备文件清单

- .py：boot.py, main.py, ht_main.py, variable.py, music.py, pen.py, scan.py
- .mpy：ST7735, basic, autoMotionOne, controlBoardAlphaPiOne, max30102, remoteControlActuatorOne, remoteControlSensorOne, steeringEngineActuatorAlphaPiOne, sysfont
- : HZK16 + gb2312（中文字库）、*.htbmp/*.bmp（图形）、*.dat（音频）、testok

11. 方法学 / 工具（均在 board_dump/）

scan_only.py	通用 raw-REPL 跑一段扫描代码并抓输出
dump_src.py	拉取设备文件；.mpy 用 ubinascii.b2a_base64 编码后取回
decode_mpy.py	解析 full_out.txt 的 base64 并落盘到 dumped/
probe_car.py	只读探测车用 I2C 0x20 电机控制器寄存器
mpy-tool.py	官方 MicroPython 反汇编器（需同目录 makeqstrdata.py）

踩坑记录：

1. 进 raw REPL 看运气（开串口会复位设备）；脚本用 `rawcmd` 命令，多次重试。
2. 大段代码一次性灌入会冲爆桥缓冲 → 改为 `START/END` 框住输出。
3. .mpy 整体 base64 在设备上会 `memory allocation failed` → 改 编码。
4. 每块 base64 `=`，整体解码会在第一个 `=` 处截断 → 必须 。
5. MicroPython 的 `bytes` `.hex()` → 用 `ubinascii.hexlify()`。

12. 待补 / 未决

- 0x20 电机控制器的 （需原理图/丝印）。
- 0x20 完整寄存器表（本文覆盖固件用到的偏移；全量探测空闲为 0）。
- `controlBoardAlphaPiOne.Update` 的 22 字节 UART 协议逐字段定义。

- `pin_map` (GPIO 33–55 映射表) 与 `WritePwm` 的 LEDC 通道对应关系。